WEEK 1 SUBMISSION • M5-A • CONTROL SUBSYSTEM

Dual Robotic Arm Autonomous Chess System

Independent Control Research

Mayar

Control Architecture | PID | Adaptive PID | Stepper Interface

Contents

1. Introduction . . . 2
2. Control Architecture Research . . . 2
3. PID Controller . . . 3
4. Conventional PID vs. Fuzzy Gain-Scheduled PID . . . 3
5. Stepper Motor Control . . . 4
6. Proposed Control Architecture . . . 4
7. Design Reasoning . . . 8
8. Dependencies and Open Design Parameters . . . 9
9. Initial Next Steps . . . 10
10. Blockers / Current Status . . . 10
11. References . . . 10

1 Introduction

The objective of the control subsystem is to ensure accurate and reliable motion of the joints of the dual robotic-arm system. Each robotic arm must follow the desired joint positions or trajectories generated by the upstream motion-planning and trajectory-generation stages.

The proposed control architecture provides a mechanism for receiving the desired joint reference, determining the difference between desired and actual position, generating a suitable motor command, driving the stepper motor through a motor driver, measuring actual position using an encoder, and feeding the measured position back to the controller.

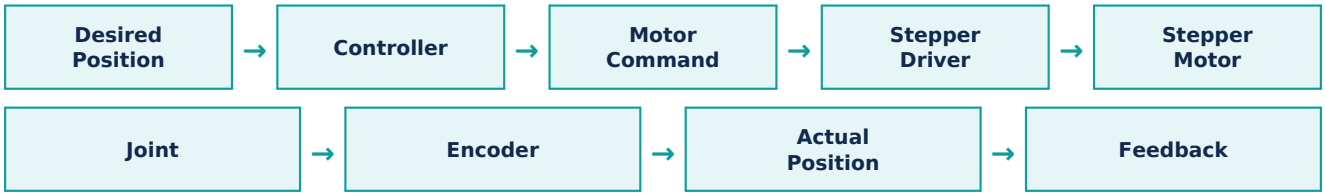
The proposed control system is based on closed-loop position control using PID control. As an adaptive enhancement, fuzzy gain scheduling is proposed to allow the PID gains to be adjusted according to the operating condition. Conventional PID is retained as the baseline for later comparison.

2 Control Architecture Research

A control system can operate in open-loop or closed-loop form. In an open-loop system, the controller generates a command without measuring the actual output. For a stepper-driven robotic joint, the controller would rely on the commanded steps and assume that the motor reaches the expected position.

In a closed-loop system, the actual output is measured and compared with the desired output. The resulting error is used by the controller to determine the corrective control action.

The proposed architecture therefore follows this signal chain:



This approach is preferred because the controller receives information about the actual joint position instead of relying only on commanded motor steps.

3 PID Controller

A PID controller is a feedback controller composed of proportional, integral, and derivative terms. It operates on the error between the desired and actual joint positions. The continuous-time PID equation is:

$$u(t) = K_p\, e(t) + K_i \int e(t)\, dt + K_d\, de(t)/dt$$

where u(t) is the controller output, e(t) is the control error, and K_p, K_i, and K_d are the proportional, integral, and derivative gains. The main PID input is the position error:

$$e(t) = q_d(t) - q(t)$$

where $q_d(t)$ is the desired joint position and q(t) is the measured actual position.

The proportional term responds to the current error. The integral term considers accumulated error and can reduce persistent steady-state error. The derivative term responds to the rate of change of the error and can help reduce excessive overshoot and oscillatory behavior. In practical implementation, derivative action may require filtering depending on encoder measurement noise.

> For a conventional PID controller, the gains are fixed after tuning. The PID output u(t) is a control output; it is **not** assumed to be a STEP pulse. A separate motion-command and step-generation layer is therefore included between the controller and the stepper-driver interface.

4 Conventional PID vs. Fuzzy Gain-Scheduled PID

A conventional PID controller uses a fixed set of K_p, K_i, and K_d values. This provides a simple and useful baseline, but fixed gains may not provide the same performance under all operating conditions.

The proposed adaptive method is fuzzy gain scheduling. The fuzzy scheduler does not replace the PID controller. Instead, it determines or modifies the PID gains according to selected scheduling inputs, while the PID remains responsible for generating the control output.

The selection is specifically motivated by the stepper-motor study by Elsodany, Rezeka, and Maharem, titled “Adaptive PID control of a stepper motor driving a flexible rotor.” The study investigates adaptive PID control for a stepper motor and uses fuzzy gain scheduling to adapt PID parameters. The reported simulation results showed improved performance compared with conventional PID for the investigated system.

> This paper supports investigating fuzzy gain scheduling for the proposed system; it does not prove that the same controller will automatically be optimal for the robotic arm. The final performance must be validated on the actual system or a representative simulation.

The exact fuzzy inputs, membership functions, rules, and gain-update formulation are not finalized in Week 1 and remain part of the detailed design stage.

5 Stepper Motor Control

A stepper motor produces rotational motion in discrete steps. For example, a 1.8° stepper motor has:

$$N = 360° / 1.8° = 200 \text{ full steps / revolution}$$

The controller normally does not drive the motor windings directly. A stepper motor driver receives the control signals and provides the required electrical drive to the motor.

The proposed controller-to-driver interface uses **STEP**, **DIR**, and **ENABLE**. STEP provides incremental motion commands, DIR determines the commanded direction, and ENABLE is used to enable or disable the driver. The exact signal polarity and timing depend on the selected driver.

Microstepping divides a full step into smaller commanded increments. For a 1.8° motor:

Full step	1/2	1/4	1/8	1/16
1.8°	0.9°	0.45°	0.225°	0.1125°

Microstep resolution should not be treated as equivalent to guaranteed absolute positioning accuracy.

6 Proposed Control Architecture

The proposed architecture is divided into seven functional layers. Figure 1 provides the overall architecture, while Figures 2–8 expand each layer individually.

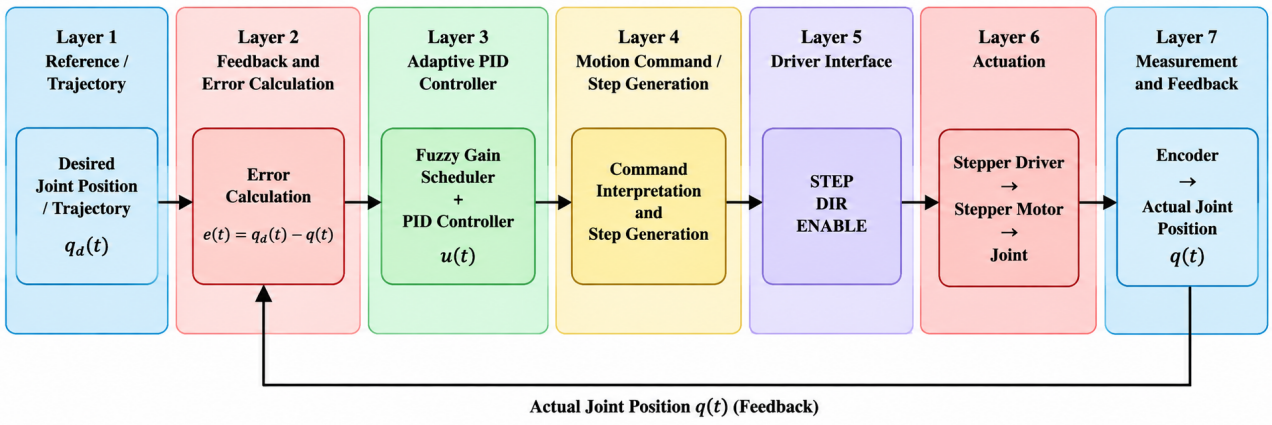


Figure 1. Overall Proposed Control Architecture

Layer 1 — Reference / Trajectory

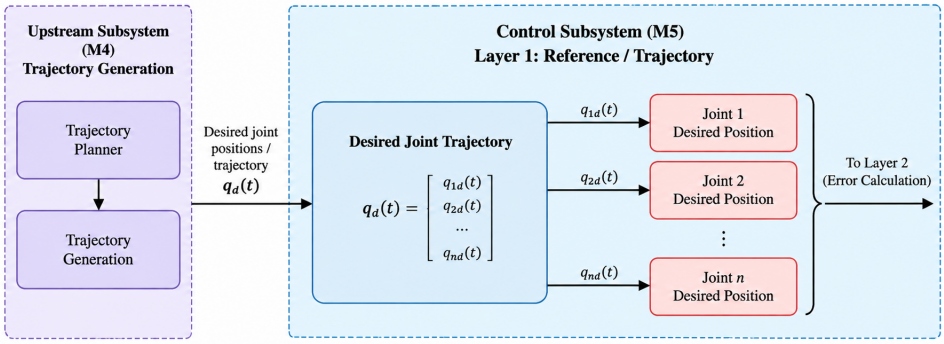


Figure 2. Layer 1: Reference / Trajectory

The control subsystem receives the desired joint position or trajectory from the upstream trajectory-generation stage. The reference may be represented as $q_d(t)$. The control subsystem does not generate the high-level trajectory itself; it receives the required joint reference and focuses on tracking it.

For multiple joints, the desired reference can be represented as a vector:

$$q_d(t) = [\, q_{1d}(t),\ q_{2d}(t),\ \ldots,\ q_{nd}(t) \,]$$

The output of this layer is passed to the error-calculation stage.

Layer 2 — Feedback and Error Calculation

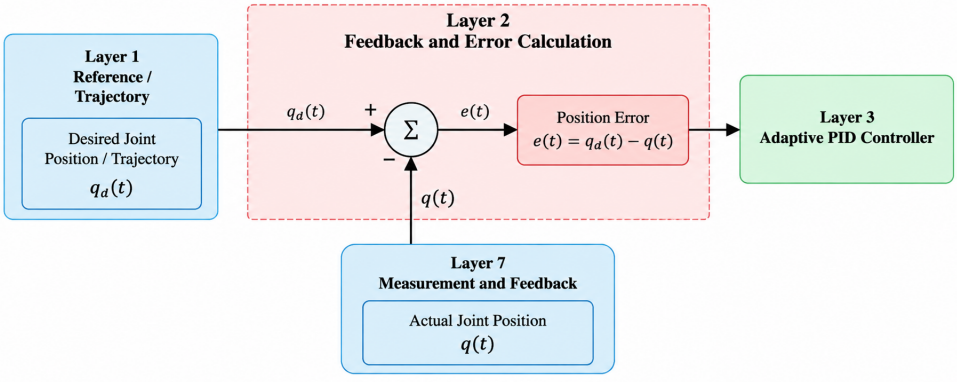


Figure 3. Layer 2: Feedback and Error Calculation

The desired joint position is compared with the actual position measured by the encoder. The position error is:

$$e(t) = q_d(t) - q(t)$$

This error is the main input to the adaptive PID controller. This layer establishes the comparison between the desired trajectory and the measured joint position.

Layer 3 — Adaptive PID Controller

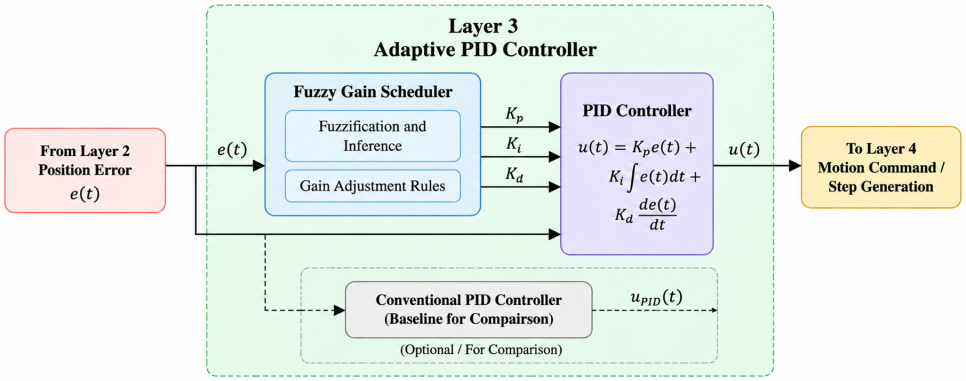


Figure 4. Layer 3:Adaptive PID Controller

This layer contains the fuzzy gain scheduler and the PID controller. The fuzzy scheduler determines the PID gains based on the selected scheduling inputs. The PID controller then uses the current gains and the position error to calculate the control output. Conceptually:

$$Fuzzy\ Scheduler \rightarrow K_p, K_i, K_d \rightarrow PID \rightarrow u(t)$$

The conventional PID controller is retained as the baseline for later comparison. The exact fuzzy structure is intentionally left open for the detailed design stage.

Layer 4 — Motion Command / Step Generation

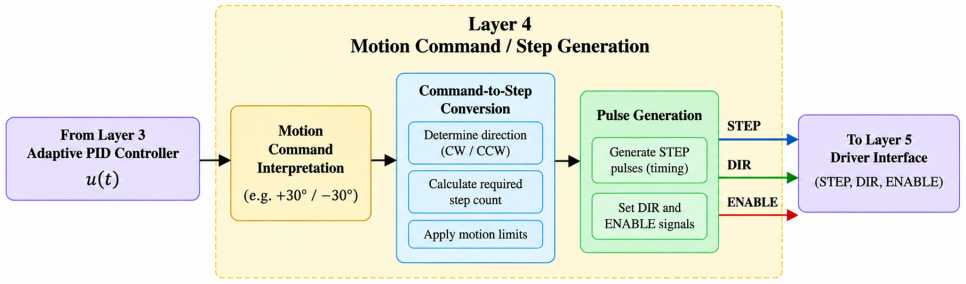


Figure 5. Layer 4: Motion Command / Step Generation

The PID control output is passed to a dedicated motion-command and step-generation stage. This layer separates the control algorithm from the electrical stepper-driver interface.

A proposed command representation may use a signed motion command, for example +30° or −30°, where the sign represents the selected positive/negative direction. This is an illustrative proposed representation, not a finalized statement that the PID output must use degrees.

The conversion layer determines direction, required step count, pulse generation, and motion limits. The exact conversion depends on the selected motor, driver, microstepping configuration, and implementation strategy.

Layer 5 — Driver Interface

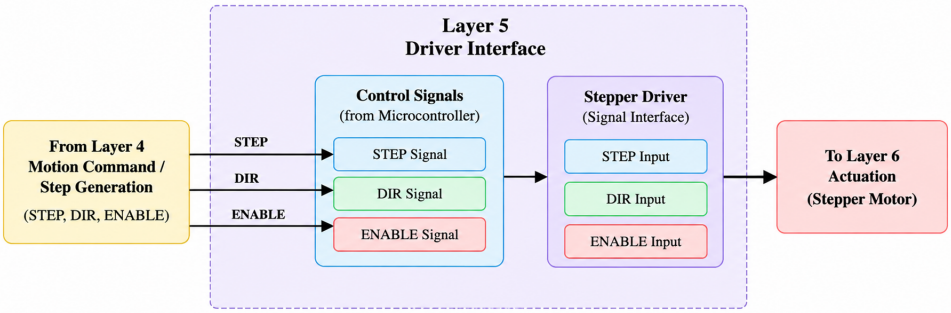


Figure 6. Layer 5: Driver Interface

The generated control signals are provided to the stepper driver through the proposed STEP, DIR, and ENABLE interface. The driver interprets these signals and provides the electrical drive required by the motor.

The exact signal polarity, timing requirements, and supported microstepping modes depend on the final driver selected by the electronics team.

Layer 6 — Actuation

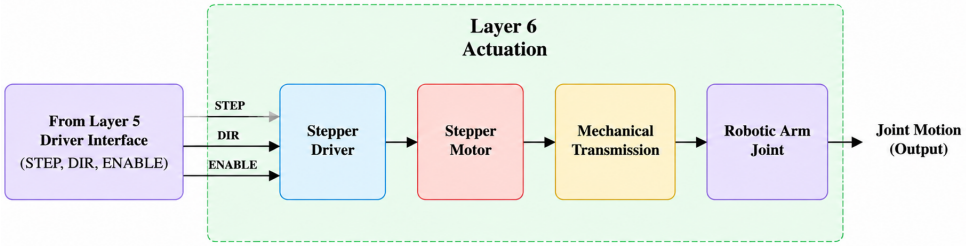


Figure 7. Layer 6: Actuation

The stepper driver supplies the electrical drive to the stepper motor. The motor generates rotational motion, which is transferred through the mechanical transmission to the corresponding robotic-arm joint.

The resulting joint motion is the physical output that the control system is designed to regulate.

Layer 7 — Measurement and Feedback

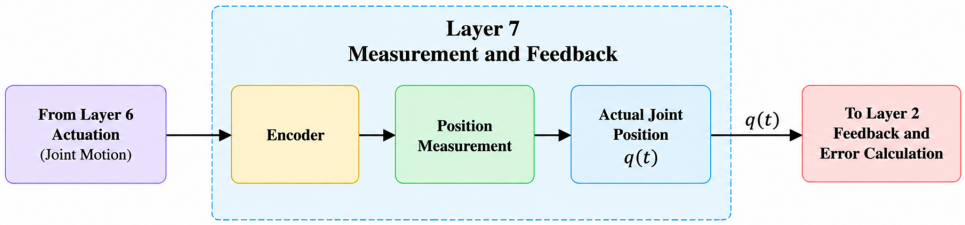


Figure 8. Layer 7: Measurement and Feedback

An encoder measures the actual motor or joint position and provides the measured position q(t). This measurement is returned to the feedback and error-calculation layer.

The feedback closes the control loop and allows the controller to determine the current position error continuously.

7 Design Reasoning

The proposed decisions are based on the project control requirements and the role assigned to M5.

1	**Closed-loop control** selected because the proposed architecture uses encoder feedback, allowing actual position to be compared with desired position.
2	**Conventional PID as baseline** selected because it provides a clear and relatively simple reference controller against which the adaptive approach can be evaluated.
3	**Fuzzy gain scheduling as adaptive enhancement** selected because it allows PID gains to vary with operating condition. The stepper-motor-specific research by Elsodany et al. provides direct motivation for investigating this approach in a stepper-motor application.
4	**STEP/DIR/ENABLE interface** selected as the proposed controller-to-driver interface, while exact timing and polarity remain dependent on the final driver.
5	**Separate command-to-step layer** included so that the PID control output is not incorrectly treated as a STEP pulse. This layer converts the controller output into a suitable motor command and then into STEP/DIR/ENABLE signals.
6	**Encoder feedback** proposed to provide actual position measurement and enable closed-loop position control. Final encoder implementation remains subject to hardware feasibility.

8 Dependencies and Open Design Parameters

The control architecture can be defined in Week 1, but several parameters must be obtained or finalized before detailed implementation.

1 Stepper motor specifications

Why needed: step angle, torque, current, voltage, and speed limits determine the physical motion capability and the conversion between angular motion and motor steps.

What it affects: step/angle conversion, maximum speed, motion resolution, and operating limits.

2 Stepper driver specifications

Why needed: STEP pulse timing, DIR behavior, ENABLE behavior, maximum step frequency, and microstepping options are driver-dependent.

What it affects: STEP/DIR generation, pulse timing, microstepping configuration, and interface implementation.

3 Encoder specifications

Why needed: the encoder provides q(t), the actual position used to calculate the error.

What it affects: feedback resolution, measurement behavior, error calculation, and closed-loop performance.

4 Mechanical parameters

Why needed: the arm links, payload, inertia, friction, transmission, and joint limits affect the dynamics of the controlled joint.

What it affects: initial PID tuning, response, overshoot, settling time, required torque, and safe motion.

5 PID output representation

Why needed: the mathematical PID output u(t) must be given a physical command representation before the step-generation layer is implemented.

What it affects: the interface between the controller and command-to-step conversion.

6 Command-to-step conversion

Why needed: a motion command must be converted into direction, step count, and pulse timing.

What it affects: motor movement, resolution, speed, and STEP/DIR generation.

7 Fuzzy gain-scheduler design

Why needed: the adaptive controller still requires decisions about fuzzy inputs, membership functions, rules, gain outputs, and limits.

What it affects: how K_p, K_i, and K_d adapt and therefore the adaptive controller behavior.

8 Controller sampling/update rate

Why needed: the closed-loop controller repeatedly reads feedback, calculates error, updates the controller, and generates a new command.

What it affects: responsiveness, computational requirements, and control behavior.

9 Motion and control limits

Why needed: controller commands must remain within motor, driver, and mechanical limits.

What it affects: output saturation, safe operation, and integral-windup handling.

9 Initial Next Steps

The immediate next steps are limited to the design handover and information required to move from the Week 1 architecture into the detailed design stage.

1	Receive the initial/final motor and driver specifications from M9, including steps/rev, current, voltage, motor limits, and driver capabilities.
2	Receive the trajectory requirements from M4, including the expected joint-position commands and motion limits.
3	Confirm the encoder configuration, if encoder feedback is implemented, including its measurement resolution and interface.
4	Use the obtained motor and driver specifications to finalize the detailed stepper-interface requirements.
5	Proceed to the detailed PID design and step/angle conversion approach in the next design stage, in coordination with M9.

These steps intentionally do not claim that final PID tuning, full fuzzy-rule design, or complete stepper implementation has already been completed. Those activities are scheduled in the later detailed-design, implementation, and testing stages of the project.

10 Blockers / Current Status

Status: No blocker prevents the Week 1 control architecture from being defined.

The main open items are hardware- and implementation-dependent: the final motor and driver specifications, encoder configuration, mechanical parameters, exact PID-output representation, command-to-step conversion, and the detailed fuzzy gain-scheduling design.

These are treated as dependencies and open design parameters rather than failures of the Week 1 architecture. The architecture is sufficiently defined to communicate the required interfaces to M4 and M9 and to proceed to the next design stage.

11 References

[1] Project Team Structure, “Dual Robotic Arm Autonomous Chess System,” Week 1–5 team plan and M5/M9 responsibilities.

[2] M5-A Week 1 Tasks, “Independent Control Research,” required Week 1 submission format.

[3] N. M. Elsodany, S. F. Rezeka, and N. A. Maharem, “Adaptive PID control of a stepper motor driving a flexible rotor,” *Alexandria Engineering Journal*, vol. 50, no. 2, pp. 127–136, 2011, doi: 10.1016/j.aej.2010.08.002.

[4] MathWorks, “Model Gain-Scheduled Control Systems in Simulink.”

[5] MathWorks, “Implement Fuzzy Gain-Scheduled PID Controllers.”

[6] MathWorks, “PID Gain Scheduler.”